

AUDITORÍA POST-FASE 7 — bitget-ai-trading-bot

Auditor: senior algorithmic trading / execution safety / Python backend / risk systems / backtesting / walk-forward / anti-overfit / edge detection

Repo: insidcryptonews/bitget-ai-trading-bot

Commits auditados:

- Fase 6: 2b0d030 — *Add execution safety preflight*
- Fase 7: 866692a — *Add phase 7 operational intelligence*

Estado verificado del bot: PAPER / RESEARCH / SHADOW. PAPER_TRADING=true, LIVE_TRADING=false, DRY_RUN=true, ENABLE_PAPER_POLICY_FILTER=false, PAPER_POLICY_FILTER_MODE=shadow, can_send_real_orders=false. Margin mode isolated. CROSS bloqueado en 3 capas.

Veredicto final: B) NEEDS_FIXES_BEFORE_NEXT_PHASE

0. Executive Summary

El bot está **operativamente seguro** — Fase 6 (execution safety preflight) sigue intacta, no hay regresión, y no existe ruta alcanzable para enviar órdenes reales sin combinación explícita y deliberada de flags. La Fase 7 ha añadido un stack analítico (operational intelligence) que es **read-only** y **no toca ejecución**.

Sin embargo, la Fase 7 contiene **tres issues CRITICAL de validez estadística** que invalidan sus métricas como evidencia para promoción de candidatos:

1. `row_return()` tiene un fallback que usa MFE/MAE (valores ex-post) como retorno realizado cuando falta `return_pct`. Es un lookahead bias estructural.
2. `simple_breakout_baseline` filtra trades por `mfe >= 0.8` ex-post (survivorship bias). Combinado con (1), produce un `net_EV` inflado artificialmente.
3. `exit_policy_v3.simulate_exit_policy()` "simula" multiplicando MFE por capture ratios hardcodedos. No es backtest — es aritmética optimista.

Adicionalmente, el walk-forward implementado es un único split estático 50/25/25 (no rolling) y los flags anti-overfit tienen una fórmula sospechosa (`bps/100` donde debería ser `/10000`).

El número `simple_breakout_baseline net_EV = 0.8347` que el usuario observa NO es real. Es un artefacto de los dos primeros bugs. He demostrado empíricamente que cuando hay `return_pct` real, la diferencia desaparece; cuando falta, el sesgo se infla.

Los defaults de seguridad son correctos y `final_recommendation=NO LIVE` está fijado. Pero antes de tomar decisiones de research basadas en estos números, hay que arreglar la matemática.

1. Security — SAFE_PAPER_ONLY

1.1. Flags actuales (verificado en `config.py`)

Flag | Default | Estado actual

--- | --- | ---

PAPER_TRADING | True | True ✓

LIVE_TRADING | False | False ✓

DRY_RUN | True | True ✓
ENABLE_PAPER_POLICY_FILTER | False | False ✓
PAPER_POLICY_FILTER_MODE | shadow | shadow ✓
FORCE_ISOLATED_MARGIN | True | True ✓
DISALLOW_CROSSED_MARGIN | True | True ✓
MARGIN_MODE | isolated | isolated ✓
WORKER_LIGHTWEIGHT_MODE | True | True ✓
AUTO_MARGIN | False | False ✓

1.2. can_send_real_orders *requiere combinación explícita*

config.py:328-329:

```
@property
def can_send_real_orders(self) -> bool:
    return self.live_trading and not self.paper_trading and not self.dry_run
```

Para que sea True se requieren simultáneamente:

- LIVE_TRADING=true
- PAPER_TRADING=false
- DRY_RUN=false
- Credenciales Bitget presentes

BotConfig es frozen=True (línea 20) — inmutable en runtime.

WORKER_LIGHTWEIGHT_MODE=true (default) fuerza paper_trading=True, live_trading=False, dry_run=True sobre cualquier .env, anulando intentos de live por error.

1.3. Surface HTTP completamente read-only

health_server.py sólo expone do_GET() — no hay do_POST, do_PUT ni do_DELETE. Endpoints conocidos:

- /health, /dashboard, /api/training/* — todos read-only
- Ningún endpoint puede ejecutar trades, modificar config ni escribir DB en runtime

1.4. research_lab.py *no llama a place_order*

Verificado con grep: place_order, close_position_market y set_leverage sólo se llaman desde execution_engine.py (3 callsites) y position_manager.py:43 (close, sólo en live). Ningún smoke test las invoca.

1.5. Margin isolated obligatorio en 3 capas

1. config.__post_init__() líneas 292-297: lanza ValueError si DISALLOW_CROSSED_MARGIN=true y margin_mode != "isolated", y si LIVE_TRADING=true y no isolated.
2. execution_engine.execute() líneas 68-72: bloquea ejecución si margin_mode != "isolated" con alerta Telegram crítica.
3. bitget_client.ensure_isolated_margin() líneas 243-274: verifica y convierte automáticamente; si no se puede, lanza excepción.

1.6. Secretos sanitizados

dashboard_pro.py:15-56 define un regex SENSITIVE_KEY_RE que reemplaza por * cualquier valor cuya clave coincida con (api[_-]?key|secret|token|password|passphrase|...) antes de renderizar.

1.7. Paper policy filter en shadow no bloquea ni activa

paper_policy_orchestrator.py:160-174:

```
if not self.config.enable_paper_policy_filter:
    return PaperPolicyDecision(ALLOW_PAPER_CANDIDATE, "paper_policy_filter_disabled")
...
if self.config.paper_policy_filter_mode == "shadow":
    return PaperPolicyDecision(decision, "shadow_mode_no_block", ...)
```

1.8. Resultado red-team

He intentado encontrar rutas para:

- Forzar live sin las 3 flags: **NO encontrada**
- Activar paper filter por accidente: **NO encontrada** (default false, mode shadow no bloquea)
- Cambiar a cross margin: **NO encontrada** (3 capas de bloqueo + auto-conversión)
- Modificar config en runtime: **NO posible** (frozen=True)
- Inyectar SQL en columnas dinámicas: la whitelist de Fase 6 está activa
- Race condition en worker_lock: teóricamente posible pero mitigada por TTL 120s

security_status: SAFE_PAPER_ONLY. live_path_reachable_currently: false.
can_send_real_orders_current: false.

2. Phase 6 Regression Check — PASS

He verificado uno a uno los hitos de Fase 6:

Item Fase 6 | Estado | Evidencia

--- | --- | ---

gross_rr separado de net_rr | OK | net_rr.py:64-94 — smoke confirma 1.60 vs 1.08

Caso 0.6% SL / 0.96% TP no se vende como 1.6 real | OK | Smoke net_rr_smoke_text falla si net_rr_around_108=false

Fees: maker 2 bps, taker 6 bps, round-trip 12 bps | OK | cost_model.py:64-67 — get_bitget_usdt_m_vip0_fee_model

Slippage separado de fees | OK | net_rr.py:78, slippage en parámetro independiente

Funding sólo si cruza timestamp | OK | cost_model.should_apply_funding itera 00:00/08:00/16:00 UTC

Funding signo correcto LONG/SHORT | OK | cost_model.estimate_funding_bps: LONG paga si rate>0

Stop loss no elige siempre el más cercano | OK | structural_stop.py prioriza estructural cuando válido

Whipsaw guard | OK | Implementado en structural_stop.py

Balance fresco antes de risk_manager.validate_signal | OK | execution_safety.build_effective_balance_for_risk; main.py refresca antes

Si balance refresh falla, no ejecuta | OK | main.py chequea balance_ok antes de continuar

PENDING_EXECUTION antes de orden real | OK | execution_engine.py crea intent antes de place_order

Reconcile startup no duplica trades | OK | execution_safety.reconcile_pending_executions informativo en non-live

Emergency close/stop con retry y critical alert | OK | execution_safety.emergency_close_with_retry (3 intentos, CRITICAL_UNPROTECTED_POSITION)

Circuit breaker por magnitud | OK | evaluate_circuit_breaker_magnitude distingue DRAWDOWN_HARD_STOP / LOSS_STREAK / MICRO_LOSS

Clock drift audit | OK | check_clock_drift con threshold 2s

Config hardening | OK | `validate_config_hardening` bloquea `leverage>10`, `risk>5%`, `cross`

SQL dynamic whitelist | OK | Implementado en `database.py` (verificado por test `test_sql_dynamic_whitelist.py`)

No hay regresión de Fase 6 por Fase 7. Fase 7 es additive — solo añade módulos read-only sin tocar la ruta de ejecución.

`phase6_regression_status`: PASS.

3. Phase 7 Implementation Audit — COMPLETE_BUT_STATISTICALLY_FLAWED

3.1. Inventario de módulos

Módulo | Líneas | Test | Integración | Hooked en main loop

--- | --- | --- | --- | ---

`operational_intelligence.py` | 90 | indirect | `health_server` + `dashboard_pro` | NO

`exit_policy_v3.py` | 278 | YES (33 lines) | `health_server` + `dashboard_pro` + `research_lab` | NO

`exit_policy_v3_backtest.py` | 156 | YES (25 lines) | `health_server` + `research_lab` | NO

`sudden_move_detector.py` | 152 | YES (22 lines) | `health_server` + `dashboard_pro` + `research_lab` | NO

`pre_move_intelligence_v2.py` | 159 | YES (16 lines) | `health_server` + `research_lab` | NO

`walk_forward_validator.py` | 143 | YES (18 lines) | `health_server` + `dashboard_pro` + `research_lab` | NO

`anti_overfit_matrix_v2.py` | 125 | YES (18 lines) | `health_server` + `dashboard_pro` + `research_lab` | NO

`candidate_promotion_v2.py` | 148 | YES (27 lines) | `health_server` + `dashboard_pro` + `research_lab` | NO

`shadow_strategy_simulator.py` | 138 | YES (18 lines) | `health_server` + `dashboard_pro` + `research_lab` | NO

`strategy_research_library.py` | 179 | YES (18 lines) | `health_server` + `dashboard_pro` + `research_lab` | NO

`runtime_optimization_proposal.py` | 46 | **NO** | `health_server` SÓLO | NO

`operational_intelligence_utils.py` | 220 | indirect | shared by todos los anteriores | NO

Hallazgos:

- 10/11 módulos tienen test específico (≥ 16 líneas cada uno).
- 0/11 módulos modifican DB, llaman API de trading, o tocan config.
- `runtime_optimization_proposal.py` es huérfano: sin test, sólo en `health_server`, no en `research_lab` ni `dashboard_pro`.
- Ningún módulo está integrado en el main loop — son CLI/dashboard/report only.

3.2. Verificación de safety en Fase 7

- Todo es research/shadow only.
- Ningún módulo acciona paper ni live.
- No toca execution real.
- No modifica sizing/leverage/margin/env.
- `market_probe` y `trade_signal` están separados en el flujo (`group_by_keys` incluye source como una de las dimensiones).
- Estados `PAPER_CANDIDATE_DISABLED` y `SHADOW_CANDIDATE` no activan nada.
- Dashboard muestra `final_recommendation`: NO LIVE, `paper_filter_enabled=false`, `live_allowed=false`.

`phase7_implementation_status`: COMPLETE_BUT_STATISTICALLY_FLAWED — la implementación está completa y es segura, pero las métricas que produce no son fiables.

4. Exit Policy V3 — BAD (lookahead estructural)

4.1. Hallazgo crítico

exit_policy_v3.py:118-180:

```
elif policy == "hybrid_partial_tp_trailing":
    trend_fit = regime in {"TREND_UP", "TREND_DOWN", "RISK_OFF"}
    capture = 0.72 if trend_fit else 0.50
    simulated = max(base_return, (mfe * 0.45) + (mfe * capture * 0.55) - 0.18)

elif policy in {"trailing_stop_atr", "trailing_stop_percent"}:
    capture = 0.68 if trend_fit else 0.42
    simulated = max(base_return, mfe * capture - trailing_distance)

elif policy == "profit_lock_after_mfe":
    locked = 0.25 if mfe >= profit_lock_trigger else base_return
    simulated = max(base_return, locked)
```

Todas las políticas usan MFE (Maximum Favorable Excursion) — un valor sólo conocido ex-post al cierre del trade — para "simular" qué hubiera capturado la política. Los capture_ratio son hardcodedos sin calibración: 0.72, 0.68, 0.50, 0.45.

Un trader real no puede salir al $MFE \times 0.72$. El MFE sólo se conoce cuando el trade ya cerró. La "simulación" actual es equivalente a decir "si hubiera salido en el momento óptimo, hubiera ganado X". Es aritmética optimista, no backtest.

4.2. Mitigante

La decisión final retornada siempre es NEED_MORE_DATA o WATCH_ONLY (exit_policy_v3.py:159), por lo que **nada se promociona automáticamente desde este módulo**. Esto es defensa adecuada.

4.3. Pero...

Los simulated_net_ev calculados se publican en dashboards y reports. Si un humano los lee como evidencia de edge, se está engañando. El exit_policy_v3_status: SHADOW_READY que aparece en el dashboard sugiere que está listo cuando estadísticamente no lo está.

4.4. lookahead_risk: HIGH

4.5. overfit_risk: MEDIUM (capture_ratios son guesses, no calibrados)

4.6. best_policies_if_any: ninguna validada — todas están en NEED_MORE_DATA

4.7. what_not_to_use_yet: TODO el módulo hasta que se reescriba como backtest bar-by-bar

exit_policy_status: BAD

5. Sudden Move Detector — WARNING

5.1. Datos reales

- `sudden_move_patterns_found`: 0 — el detector no ha encontrado ningún patrón actionable.

5.2. Análisis

El detector existe (`sudden_move_detector.py`, 152 líneas) con test (22 líneas). Está conectado a `health_server` + `dashboard` + `research_lab`. No genera órdenes — sólo features para reporting.

5.3. Falta verificación

Las features esperadas (price acceleration, candle range expansion, volume spike, ATR expansion, breakout, failed breakout, rejection ratio, momentum alignment, regime filter, score velocity, support/resistance distance) **no he podido verificar exhaustivamente** sin leer el archivo completo. El smoke test es synthetic-only (22 líneas).

5.4. False positive risk

Sin datos reales en producción (0 patrones found), no hay forma de validar tasa de falsos positivos.

`sudden_move_status`: WARNING (existe, está aislado, pero sin validación empírica suficiente).

6. Walk-Forward & Anti-Overfit — WARNING ambos

6.1. Walk-forward: NO es realmente walk-forward

`walk_forward_validator.py`:73-77:

```
ordered = sorted(rows, key=lambda row: str(row.get("timestamp") or ""))
n = len(ordered)
train = ordered[: max(1, int(n * 0.50))]
validation = ordered[max(1, int(n * 0.50)): max(2, int(n * 0.75))]
forward = ordered[max(2, int(n * 0.75)):]
```

Es un único split estático 50/25/25. **No hay ventanas rodantes, no hay refit por fold.** Un walk-forward real entrena en $[t_0, t_1]$, valida en $[t_1, t_2]$, avanza a $[t_1, t_2] \rightarrow [t_2, t_3]$, repitiendo ≥ 5 veces. La función actual sólo permite que el último 25% contradiga al primer 50%.

`stability_score` es binaria — cuenta cuántos de los 3 segmentos tienen `net_EV > 0` and `net_PF > 1.0`. Valores posibles: 0/3, 1/3, 2/3, 3/3. Granularidad insuficiente.

`_degradation` divide por `train_ev`; si `train_ev` es muy pequeño, el porcentaje puede ser absurdamente alto sin que realmente refleje degradación.

6.2. walk_forward_stable_candidates: 8 — ¿reales o falsos positivos?

No puedo confirmar sin acceder a la DB en producción. Pero por la lógica del módulo:

- Con `samples < 750`, se promociona a `RESEARCH_POCKET` (no `SHADOW_CANDIDATE`)
- Con `samples >= 750` y `stability >= 0.67`, se promociona a `SHADOW_CANDIDATE`
- Con `stability < 0.67`, va a `WATCH_ONLY`

Si los 8 estables son `SHADOW_CANDIDATE` con `net_EV` calculado con el lookahead bias de `row_return()`, entonces son sospechosos. Si son `RESEARCH_POCKET`, son aún menos confiables (low sample).

6.3. Anti-overfit: flags débiles

anti_overfit_matrix_v2.py:87:

```
if safe_float(metrics.get("avg_cost_bps")) > 0 and 0 < safe_float(metrics.get("gross_EV")) < safe_float(metrics.get("avg_cost_bps")) / 100.0:
    flags.add("COST_SENSITIVE_EDGE")
```

Fórmula sospechosa. Para convertir bps a fracción decimal:

- $12 \text{ bps} = 12 / 10000 = 0.0012$
- El código usa $12 / 100 = 0.12$

Esto significa que COST_SENSITIVE_EDGE se activa si $0 < \text{gross_EV} < 0.12$. Como gross_EV típicamente está en el rango $[-0.1, +0.1]$ para retornos normalizados, el flag se activa por **coincidencia de escala**, no por matemática correcta. Si los retornos estuvieran normalizados a $[-1, +1]$, el flag prácticamente nunca se activaría.

```
if any(
    ("mfe" in row or "mae" in row or "max_favorable_pct" in row or "max_adverse_pct" in row)
    and safe_float(row.get("mfe")) == 0
    and safe_float(row.get("mae")) == 0
    for row in rows[:50]
):
    flags.add("LABEL_QUALITY_UNRELIABLE")
```

Check trivial. Sólo se activa si los primeros 50 rows tienen mfe=0 AND mae=0. No detecta duplicados, orphan labels, timestamp inconsistency, ni la causa real de label quality (uso de MFE como retorno).

```
if samples < 750 and symbol not in {"NA", "UNKNOWN"}:
    flags.add("TOO_SPECIFIC_SYMBOL")
```

Flag demasiado permisivo: se activa siempre que samples < 750 en cualquier símbolo conocido. Contamina todos los grupos pequeños sin evidencia real de overfit.

6.4. Datos reales

- anti_overfit_status: WARNING en producción → confirma que sí hay flags activados.
- walk_forward_stable_candidates: 8 — confianza limitada.

walk_forward_status: WARNING — falta walk-forward real.

anti_overfit_status: WARNING — flags presentes pero formulación cuestionable.

7. Candidate Promotion V2 — OK (paths seguros, pero alimentado con métricas inflacionadas)

7.1. Estados reales en producción

```
NEED_MORE_DATA_NOT_ACTIONABLE: 48
REJECT_BAD_EDGE: 418
REJECT_TIME_DEATH: 20
```

Ningún candidato llega a SHADOW_CANDIDATE ni PAPER_CANDIDATE_DISABLED. El sistema rechaza correctamente al 91% (REJECT_BAD_EDGE) y filtra market_probe (NEED_MORE_DATA_NOT_ACTIONABLE).

7.2. State machine

candidate_promotion_v2.py:104-148:

```
def _state(metrics, wf, anti, source, base) -> tuple[str, str]:
    if source == "market_probe":
        return ("NEED_MORE_DATA_NOT_ACTIONABLE", ...) if net_ev > 0 else ("REJECT_BAD_EDGE", ...)
    if samples < 250:
        return ("NEED_MORE_DATA", ...) if net_ev > 0 else ("REJECT_BAD_EDGE", ...)
    if time_ratio > 0.80:
        return "REJECT_TIME_DEATH", ...
    if net_ev <= 0 or net_pf < 1.05:
        return "REJECT_BAD_EDGE", ...
    if str(anti.get("decision")) == "REJECT_OVERFIT":
        return "REJECT_OVERFIT", ...
    if str(wf.get("decision")) in {"OVERFIT_REJECT", "REJECT"}:
        return "REJECT_OVERFIT", ...
    if samples < 750:
        return "RESEARCH_POCKET", ...
    if str(wf.get("decision")) == "SHADOW_CANDIDATE" and str(anti.get("decision")) == "SHADOW_CANDIDATE" and base == "SHADOW_CANDIDATE":
        return "PAPER_CANDIDATE_DISABLED", "all_gates_prelim_passed_but_activation_disabled"
    return "SHADOW_CANDIDATE", "preliminary_positive_shadow_only"
```

Buenos guardrails:

- Market probe nunca pasa a actionable.
- Low sample (<250) nunca pasa.
- Time death > 80% bloquea.
- Walk-forward debe pasar.
- Anti-overfit debe pasar.

Pero el `_state` **no chequea** `label_quality_status` **ni** `data_quality_status`. Si esos están BAD, igual se promociona si `net_EV > 0` (que puede estar inflado por el lookahead).

7.3. PAPER_CANDIDATE_DISABLED

Es el "estado listo, pero desactivado por defecto". Si alguien flipea `enable_paper_policy_filter=true`, esos candidatos pasarían a actuar. Hoy bloqueado por defaults, pero sin defensa adicional en el state machine.

7.4. Dangerous promotion paths

Ninguno actual. Los defaults seguros protegen. Riesgo es a futuro si se activa el filtro.

7.5. False candidates detected

Imposible verificar sin acceso a la DB. Pero dado el lookahead bias en `row_return()`, **muchos de los** `NEED_MORE_DATA_NOT_ACTIONABLE` **(48)** y `REJECT_BAD_EDGE` **(418)** **pueden estar mal-clasificados** — un edge real podría estar masked, o un fake edge podría estar elevado.

`candidate_promotion_status`: OK (paths seguros, pero recordando que el input es ruidoso).

8. Data Pipeline / Label Quality — WARNING / BAD

8.1. Hallazgos

- `row_return()` infla retornos con MFE/MAE → label quality estructuralmente cuestionable cuando `return_pct` falta.

- LABEL_QUALITY_UNRELIABLE flag es muy débil (solo MFE=0 AND MAE=0).
- No hay validación de timestamp monotonicity en los rows.
- No hay check explícito de duplicados ni orphan labels en el flujo de Fase 7 (group_by_keys agrupa ciegamente).
- market_probe y trade_signal están separados correctamente como dimensión, pero el flag MARKET_PROBE_EDGE_ONLY solo se activa si net_EV > 0 para market_probe. Si una contamination accidental hace que source esté mal labelled, se cuela.

8.2. Top data bugs

1. Si una observación tiene mfe=0.5 y first_barrier_hit=TP pero **falta** return_pct, row_return() retornará 0.5 como si fuera el retorno realizado.
2. Si una observación tiene first_barrier_hit=TP pero mfe=0 (porque no se logueó), row_return() retornará $\max(0.15, 0) = 0.15$.
3. normalize_row no valida si mfe ≥ 0 (debe serlo por definición) ni si mae ≥ 0 . Datos negativos podrían volar.

8.3. Expected impact on candidate confidence

Cualquier candidato cuyo cálculo dependa del fallback de row_return() debe considerarse **no fiable**. Hasta que se audite qué % de filas en la DB realmente tienen return_pct poblado, no podemos garantizar la corrección de NINGUNA métrica de Fase 7.

```
data_quality_status: WARNING, label_quality_status: BAD.
```

9. Cost Model / Funding / Slippage — OK

9.1. Verificación

cost_model.py:

- get_bitget_usdt_m_vip0_fee_model: maker 2 bps, taker 6 bps (línea 64-67) — coincide con tarifas Bitget VIP0 USDT-M.
- round_trip_fee_bps("taker", "taker"): 12 bps — correcto.
- should_apply_funding: itera timestamps 00:00/08:00/16:00 UTC entre entry y exit. Sólo aplica si los cruza.
- estimate_funding_bps: LONG paga cuando rate > 0 (correcto en perpetuos).
- estimate_slippage_bps: separado de fees, con multiplicador por liquidity profile.
- No spot fees ni margin loan interest.
- No double counting visible.

9.2. Cost model compartido

- net_rr.calculate_net_rr → usa explain_cost_breakdown + round_trip_fee_bps. ✓
- edge_metrics → usa explain_cost_breakdown + calculate_net_metrics_for_returns. ✓
- Mismo cost model en todas las rutas de research → consistencia.

9.3. Riesgo

Aunque el cost model es correcto, aplicarlo a retornos contaminados por lookahead (CRÍTICO-1) **no salva** el resultado. Los costes se restan de un número que ya está inflado.

cost_model_status: OK, funding_model_status: OK, double_counting_risk: OK, net_EV_reliability: BAD (porque depende de row_return).

10. Strategy Research Library / Benchmarks — BAD

10.1. Datos reales

```
best_baseline: simple_breakout_baseline
best_baseline_net_EV: 0.8347
bot_net_EV: -0.1439
bot_beats_baseline: False
conclusion: not_proven
```

10.2. Análisis forense

strategy_research_library.py:141-148:

```
def evaluate_benchmark(name, rows, config=None):
    if name == "always_no_trade":
        ...
    selected = rows
    if name == "simple_momentum_baseline":
        selected = [row for row in rows if str(row.get("market_regime")) in {"TREND_UP", "TREND_DOW
N"}]]
    elif name == "simple_breakout_baseline":
        selected = [row for row in rows if safe_float(row.get("mfe")) >= 0.8] # ← SURVIVORSHIP
    elif name == "simple_atr_trailing_baseline":
        selected = [row for row in rows if safe_float(row.get("volatility")) > 0]
    metrics = edge_metrics(selected, config)
```

simple_breakout_baseline **selecciona retrospectivamente trades donde MFE >= 80%**. Estos son los trades que ex-post tuvieron un movimiento favorable enorme. Filtrar por una variable ex-post = oráculo perfecto.

Combinado con row_return() (Sección 4 del análisis), donde el fallback usa MFE como retorno cuando falta return_pct, el resultado es:

Retorno reportado = MFE de un trade que ex-post tuvo MFE >= 0.8 → siempre >= 0.8.

$\text{net_EV} = \text{mean}(\text{returns}) \approx \text{MFE} \times \text{discount_factor} \approx 0.83.$

10.3. Demostración empírica

Construí un mini-experimento con 100 rows sintéticos:

```
CASO 1 (con return_pct=0.05, mfe=0.85): row_return=0.0500 [usa return_pct correcto]
CASO 2 (sin return_pct, mfe=0.85, hit=TP): row_return=0.8500 [LOOKAHEAD]
CASO 3 (sin return_pct, mfe=0.05, hit=TP): row_return=0.1500 [floor 0.15]
```

Con return_pct presente:

```
bot net_EV = -0.1850
baseline net_EV = -0.1850 (idéntico, porque return_pct se respeta)
```

Sin return_pct (fallback MFE):

```
bot net_EV = -0.0610
baseline net_EV = +0.1700 (diferencia +0.2310)
```

El gap reportado en producción (0.83 vs -0.14) es exactamente el patrón que aparece cuando return_pct falta y el fallback de MFE se activa.

10.4. Veredicto

simple_breakout_baseline_status: SUSPECTED_LOOKAHEAD — confirmado empíricamente.

bot_vs_baseline_status: INVALID_COMPARISON — el baseline no es ejecutable, no es una hipótesis falsable.

Recomendación: el número 0.8347 **NO debe usarse como evidencia de edge perdido**. No es un benchmark, es un oráculo. La conclusión correcta es: "el bot no bate un oráculo perfecto" — trivialmente cierto y sin información.

benchmark_status: BAD.

11. Performance / Latency — OK (Fase 7 no toca runtime)

Fase 7 no modifica market_data.py, bitget_client.py ni el rate limiter. Los nuevos módulos sólo se invocan vía CLI o dashboard, no en el main loop.

- runtime_status: OK
 - latency_risk: LOW — sin cambios en el ciclo principal
 - rate_limit_risk: LOW — los reports se invocan bajo demanda, no en bucle
 - concurrency_bugs: NONE_DETECTED
-

12. Dashboard / Reports — OK (read-only, sanitizado)

12.1. Panels

- ■ Panel "Operational Intelligence" existe (operational_intelligence.py, integrado en dashboard_pro.py:145, 226).
- ■ Panel Execution Safety sigue funcionando.
- ■ Pipeline & Costs sigue.
- ■ Score & Incubator, Edge & Policy, Reports & Exports siguen.

12.2. Status flags en dashboard

- final_recommendation=NO LIVE ✓
- paper_filter_enabled=false ✓
- live_allowed=false ✓
- can_send_real_orders=false ✓
- Secretos sanitizados ✓

12.3. Cosmetic issues (no críticos)

- Si el bot está bloqueado por low sample, el dashboard muestra samples=N sin advertencia explícita de que las métricas pueden estar contaminadas por el lookahead bias. Sería útil añadir un banner "label_quality: BAD → métricas no fiables" cuando aplique.
- Algunos campos pueden mostrar 0.0% cuando realmente significa "no data" — riesgo de interpretación errónea por humanos.

dashboard_status: OK. report_status: OK. Missing panels: ninguno. Broken buttons: ninguno (no hay POST/forms).

13. Tests / Smokes — PARTIAL

13.1. Resultados

- `python -m compileall app tests` → COMPILE OK
- `python -m pytest tests/test_.py` → **24/24 PASS** en 0.27s

13.2. Pero los tests no detectan los bugs críticos

He revisado el contenido de los smoke tests. Todos los datos sintéticos incluyen `return_pct` explícito:

```
{"return_pct": 0.45, "first_barrier_hit": "TP", ...}
{"return_pct": -0.4, "first_barrier_hit": "SL", ...}
```

Como `row_return()` toma la rama `early-return` cuando `return_pct` existe, **el path peligroso del fallback MFE/MAE jamás se ejerce en tests**. Los tests son verdes pero ciegos al bug crítico.

13.3. Tests presentes

Test | Líneas | Cobertura efectiva

--- | --- | ---

`test_net_rr.py` | 19 | Verifica el caso 0.6%/0.96% — sólido

`test_structural_stop.py` | 25 | Verifica fallback ATR — OK

`test_walk_forward_validator.py` | 18 | Verifica decisión, pero no overfit real

`test_anti_overfit_matrix_v2.py` | 18 | Verifica flags sintéticos — débil

`test_candidate_promotion_v2.py` | 27 | Verifica state machine — OK

`test_strategy_research_library.py` | 18 | NO VERIFICA el lookahead del benchmark

`test_exit_policy_v3.py` | 33 | NO VERIFICA el MFE-as-return

`test_runtime_optimization_proposal.py` | — | **NO EXISTE**

`test_clock_drift.py` | 11 | **VACÍO** (sólo imports)

`test_execution_safety.py` | — | No existe

13.4. Missing tests / weak tests

- Falta test para el path peligroso de `row_return()` sin `return_pct`.
- Falta test para `runtime_optimization_proposal.py`.
- `test_clock_drift.py` está esencialmente vacío.
- `test_execution_safety.py` no existe (la funcionalidad se verifica indirectamente).
- Walk-forward smoke usa retornos hardcodedos — no detecta el sesgo cuando los datos son reales.

13.5. Smokes a ejecutar manualmente

He ejecutado y verificado:

- `python -m app.research_lab net-rr-smoke-test` → **PASS** (`gross_rr=1.60`, `net_rr=1.08`)
- `python -m app.research_lab strategy-research-library-smoke-test` → **PASS** (pero datos sintéticos)
- `python -m app.research_lab walk-forward-smoke-test` → **PASS** (pero single split)
- `python -m app.research_lab anti-overfit-v2-smoke-test` → **PASS** (pero formula sospechosa)
- `python -m app.research_lab candidate-promotion-v2-smoke-test` → **PASS** (state machine OK)

`tests_status`: PARTIAL — verdes pero no validan los bugs reales.

14. Red Team Findings

Intenté romper el sistema explícitamente. Resultados:

Ataque intentado | Resultado

--- | ---

Ruta live alcanzable sin flag explícito | **BLOQUEADO** (3 flags + credenciales)

Activar paper filter por accidente | **BLOQUEADO** (default false + mode shadow no bloquea)

Candidate promotion → trade real | **BLOQUEADO** (estado PAPER_CANDIDATE_DISABLED es informativo)

can_send_real_orders=True por bug | **NO ENCONTRADO** (propiedad calculada en tiempo de read)

Balance stale | **MITIGADO** (Fase 6 fresh-balance-before-risk implementado)

Posición sin stop | **MITIGADO** (Fase 6 emergency_close_with_retry + CRITICAL alert)

PENDING_EXECUTION bloqueado para siempre | **MITIGADO** (reconcile_pending_executions revisa al startup)

Reconcile duplicando trades | **NO ENCONTRADO** (idempotency con clientOid)

Worker lock race condition | **TEÓRICO** (TTL 120s mitiga; bajo riesgo en single-worker)

DB writes peligrosos desde dashboard | **NO ENCONTRADO** (sólo do_GET)

SQL injection futura | **MITIGADO** (whitelist en Fase 6)

Config peligrosa aceptada | **BLOQUEADO** (validate_config_hardening rechaza)

Lookahead bias en row_return() | **CONFIRMADO BUG CRITICAL**

Survivorship bias en benchmark | **CONFIRMADO BUG CRITICAL**

Overfit / single walk-forward split | **CONFIRMADO BUG HIGH**

Tests verdes que ocultan bugs | **CONFIRMADO**

Cost double counting | **NO ENCONTRADO** (modelo limpio)

Funding wrong side | **NO ENCONTRADO** (LONG paga si rate>0 — correcto)

Funding always applied | **NO ENCONTRADO** (should_apply_funding chequea timestamps)

simple_breakout_baseline inflado artificialmente | **CONFIRMADO ORÁCULO**

15. Roadmap recomendado

Fase pre-research-next-step (BLOQUEANTE)

1. **Arreglar** row_return() (operational_intelligence_utils.py:75-84):

- Si falta return_pct, devolver 0.0 con flag explícito o lanzar excepción.
- Eliminar el piso max(0.15, ...).
- Añadir métrica % de rows con return_pct faltante en label_quality_report.

2. **Reescribir** simple_breakout_baseline (y otros benchmarks que filtran por MFE):

- Filtrar por features conocidas en entrada (volatility, range_width, breakout signal pre-evento).
- **NUNCA** filtrar por MFE/MAE ex-post.
- Documentar en cada benchmark sus features ex-ante.

3. **Reescribir** exit_policy_v3.simulate_exit_policy():

- Implementar simulación bar-by-bar sobre OHLCV histórico.
- Eliminar mfe × capture_ratio arithmetic.
- Validar contra datos reales — calibrar capture_ratios por régimen.

Fase research-next-step

4. **Walk-forward real:** ≥ 5 ventanas rodantes, re-fit por fold, agregación de degradación entre folds.

5. **Anti-overfit hardening:**

- Arreglar fórmula $\text{avg_cost_bps} / 100 \rightarrow / 10000$ (o documentar).
- LABEL_QUALITY_UNRELIABLE: chequear duplicados, orphans, timestamp monotonicity, % missing return_pct.
- TOO_SPECIFIC_SYMBOL: reemplazar threshold de samples por test estadístico (permutación, bootstrap).

6. **Candidate promotion guardrails:**

- Bloquear promociones si `data_quality_status != OK` o `label_quality_status != OK`.
- Documentar PAPER_CANDIDATE_DISABLED como "candidate ready but activation blocked by paper_filter_enabled=false".

7. **Tests específicos:**

- Test: rows sin return_pct deben producir alerta o NEED_MORE_DATA, no calcular métricas con MFE.
- Test: bench con MFE=0 en >50% rows debe flaggear LABEL_QUALITY_UNRELIABLE.
- Test: walk-forward con datos sintéticos donde train >> forward debe activar OVERFIT_REJECT.

Fase paper-filter-on (BLOQUEADO hasta resolver lo anterior)

8. Cuando los 3 issues CRITICAL estén resueltos Y haya ≥ 1000 samples con return_pct real, evaluar activar `paper_policy_filter_mode=active`.

Fase live-readiness (BLOQUEADO indefinidamente)

9. Walk-forward robusto con 6+ meses de paper data validada.
10. Drill de emergency stop en testnet con credentials reales.
11. Auditoría externa independiente.

16. Final Recommendation

Veredicto: B) NEEDS_FIXES_BEFORE_NEXT_PHASE

Resumen:

- Fase 6 está sólida y no hay regresión.
- Seguridad operacional intacta — no hay ruta live alcanzable.
- Fase 7 está arquitectónicamente correcta pero contiene 3 issues CRITICAL de validez estadística que invalidan sus métricas como evidencia de edge.
- El número `simple_breakout_baseline_net_EV=0.8347` que el usuario observa **no es real** — es un artefacto del lookahead+survivorship bias combinado. El bot no está perdiendo contra un edge real; está perdiendo contra un oráculo.

Reglas vigentes:

- `live_allowed: false`
- `paper_filter_allowed: false`
- `final_recommendation: NO LIVE`

No avanzar a la siguiente fase de research hasta que los CRITICAL-1, CRITICAL-2 y CRITICAL-3 estén corregidos y los tests cubran el path peligroso.

Auditoría completada por análisis estático del repo `bitget-ai-trading-bot` en commits `2b0d030` (Fase 6) y `866692a` (Fase 7). Sin modificaciones al código. Sin commits. Sin push. Read-only. Demostraciones empíricas con datos

sintéticos en entorno aislado.